

DATE	27 – 06 – 2025
PROGRAM NO.: 01	Program to implement Best First Search

SOURCE CODE:

```
from queue import PriorityQueue

def best_first_search(graph, start, goal):
    """
    Best-First Search prioritizes nodes that appear to be closest to the goal
    using a heuristic (in this case, just the edge weights).
    It doesn't guarantee the shortest path but is often fast.
    """

    visited = set()
    pq = PriorityQueue()
    pq.put((0, start))
    path = []
    total_cost = 0

    print("\n ════════════════════════════════════════════════════════════")
    print(" ||   Best-First Search (Greedy)   || ")
    print(" ════════════════════════════════════════════════════════════ ")

    print("\nGraph Connections:")
    for node in graph:
        print(f" {node} → {graph[node]}")

    while not pq.empty():
        cost, node = pq.get()
        if node in visited:
            continue

        print(f"\nVisiting: {node} (Current Cost: {cost})")
        visited.add(node)
```

```
path.append(node)
total_cost += cost

if node == goal:
    print("\n Goal Node Reached!")
    break

for neighbor, weight in graph[node]:
    if neighbor not in visited:
        print(f" Adding neighbor: {neighbor} with weight {weight}")
        pq.put((weight, neighbor))

print("\n===== Results =====")
print(f'Path: {' → '.join(path)}')
print(f'Total Cost: {total_cost}')
print("Note: Best-First doesn't guarantee optimal path!")

graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],
    'E': [],
    'F': []
}
best_first_search(graph, 'A', 'F')
```

OUTPUT:

Best-First Search (Greedy)

Graph Connections:

```
A → [('B', 1), ('C', 3)]
B → [('D', 3), ('E', 1)]
C → [('F', 5)]
D → []
E → []
F → []
```

Visiting: A (Current Cost: 0)

Adding neighbor: B with weight 1

Adding neighbor: C with weight 3

Visiting: B (Current Cost: 1)

Adding neighbor: D with weight 3

Adding neighbor: E with weight 1

Visiting: E (Current Cost: 1)

Visiting: C (Current Cost: 3)

Adding neighbor: F with weight 5

Visiting: D (Current Cost: 3)

Visiting: F (Current Cost: 5)

Goal Node Reached!

===== Results =====

Path: A → B → E → C → D → F

Total Cost: 13

Note: Best-First doesn't guarantee optimal path!

DATE	27 – 06 -2025
PROGRAM NO.: 02	Program to implement A* Algorithm

SOURCE CODE:

```

from queue import PriorityQueue

def a_star_search(graph, start, goal, h):
    """
    A* Search combines actual path cost (g) with heuristic estimate (h)
    to find the optimal path. It's guaranteed to find the shortest path
    if the heuristic is admissible (never overestimates).
    """
    open_set = PriorityQueue()
    open_set.put((h[start], start))
    g = {start: 0}
    parent = {start: None}
    visited = set()

    print("\n=====")
    print("||      A* Search      ||")
    print("=====")

    print("\nGraph Connections:")
    for node in graph:
        print(f" {node} → {graph[node]}")

    print("\nHeuristic Values (h):", h)

    while not open_set.empty():
        f, current = open_set.get()
        if current in visited:
            continue

        print(f"\nVisiting: {current} (F={f}, G={g[current]}, H={h[current]})")

```

```

visited.add(current)

if current == goal:
    break

for neighbor, cost in graph[current]:
    tentative_g = g[current] + cost
    if neighbor not in g or tentative_g < g[neighbor]:
        g[neighbor] = tentative_g
        f = tentative_g + h[neighbor]
        open_set.put((f, neighbor))
        parent[neighbor] = current
        print(f" Updated {neighbor}: G={tentative_g}, F={f}")

path = []
node = goal
total_cost = g[goal]
while node:
    path.append(node)
    node = parent[node]
path.reverse()

print("\n===== Results =====")
print(f"Optimal Path: {' → '.join(path)}")
print(f"Total Cost: {total_cost}")
print("Note: A* guarantees optimal path with admissible heuristic!")

# Example usage
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],

```

```
'E': [],
'F': []
}
```

```
h = {'A': 5, 'B': 4, 'C': 3, 'D': 2, 'E': 1, 'F': 0}
```

```
a_star_search(graph, 'A', 'F', h)
```

OUTPUT:

A* Search

Graph Connections:

```
A → [('B', 1), ('C', 3)]
B → [('D', 3), ('E', 1)]
C → [('F', 5)]
D → []
E → []
F → []
```

Heuristic Values (h): {'A': 5, 'B': 4, 'C': 3, 'D': 2, 'E': 1, 'F': 0}

```
Visiting: A (F=5, G=0, H=5)
  Updated B: G=1, F=5
  Updated C: G=3, F=6
```

```
Visiting: B (F=5, G=1, H=4)
  Updated D: G=4, F=6
  Updated E: G=2, F=3
```

```
Visiting: E (F=3, G=2, H=1)
```

```
Visiting: C (F=6, G=3, H=3)
  Updated F: G=8, F=8
```

```
Visiting: D (F=6, G=4, H=2)
```

```
Visiting: F (F=8, G=8, H=0)
```

===== Results =====

Optimal Path: A → C → F

Total Cost: 8

Note: A* guarantees optimal path with admissible heuristic!

DATE	09 – 07 – 2025
PROGRAM NO.: 03	Program to implement AO* Algorithm

SOURCE CODE:

```

def ao_star(node, graph, heuristic, path_map):
    if not graph[node]: # Leaf node
        return heuristic[node], [node]

    options = []
    for option in graph[node]:
        total_cost = 0
        total_path = []
        for child in option:
            cost, sub_path = ao_star(child, graph, heuristic, path_map)
            total_cost += cost
            total_path.extend(sub_path)
        options.append((total_cost, option, total_path))

    # Choose the best option (min cost)
    best_cost, best_children, best_path = min(options, key=lambda x: x[0])
    heuristic[node] = best_cost
    path_map[node] = (best_children, best_path)
    print(f"Best path from {node}: {best_children}, Cost: {best_cost}")
    return best_cost, [node] + best_path

# Define the AND-OR graph
graph = {
    'A': [('B'), ('C', 'D')],
    'B': [('E', 'F'), ('G')],
    'C': [],
    'D': [],
    'E': [('H')],

```

```
'F': [(T)],
'G': [],
'H': [],
'T': []
}

# Initial heuristic values (for leaf nodes only)
heuristic = {
    'A': None,
    'B': None,
    'C': 3,
    'D': 4,
    'E': None,
    'F': None,
    'G': 6,
    'H': 2,
    'T': 1
}

# This will store the best subpaths chosen during the AO* process
path_map = {}

# Start AO* from node 'A'
cost, final_path = ao_star('A', graph, heuristic, path_map)

print("\n✓ Optimal Path:", " -> ".join(final_path))
print("□ Total Cost:", cost)
```

OUTPUT:

```
Best path from E: ('H',), Cost: 2
Best path from F: ('I',), Cost: 1
Best path from B: ('E', 'F'), Cost: 3
Best path from A: ('B',), Cost: 3

✅ Optimal Path: A -> B -> E -> H -> F -> I
📊 Total Cost: 3
```

DATE	09 – 07 – 2025
PROGRAM NO.: 04	Program to Hill Climbing Algorithm

SOURCE CODE:

```
#Hill Climbing Algorithm for block world
```

```
import random
```

```
class BlockWorld:
```

```
    def __init__(self, blocks, goal):
```

```
        self.blocks = blocks
```

```
        self.goal = goal
```

```
    def get_possible_moves(self):
```

```
        moves = []
```

```
        for i in range(len(self.blocks)):
```

```
            for j in range(len(self.blocks)):
```

```
                if i != j:
```

```
                    new_state = self.blocks[:]
```

```
                    new_state[i], new_state[j] = new_state[j], new_state[i]
```

```
                    moves.append(new_state)
```

```
        return moves
```

```
    def evaluate(self):
```

```
        return sum(1 for i in range(len(self.blocks)) if self.blocks[i] == self.goal[i])
```

```
    def is_goal_state(self):
```

```
        return self.blocks == self.goal
```

```
def hill_climbing(block_world):
```

```
    current_state = block_world.blocks
```

```
    current_score = block_world.evaluate()
```

```
    step = 0
```

```

print(f"Step {step}: {current_state}, Cost: {current_score}")

while not block_world.is_goal_state():
    step += 1
    possible_moves = block_world.get_possible_moves()
    next_move, next_score = None, current_score

    for move in possible_moves:
        block_world.blocks = move
        score = block_world.evaluate()
        if score > next_score:
            next_move, next_score = move, score

    if next_move is None:
        break

    current_state, current_score = next_move, next_score
    block_world.blocks = current_state

    print(f"Step {step}: {current_state}, Cost: {current_score}")

return block_world.blocks

initial_state = ['b', 'c', 'd', 'a']
goal_state = ['a', 'b', 'c', 'd']

block_world = BlockWorld(initial_state, goal_state)

print("Initial State:", block_world.blocks)
print("Goal State:", goal_state)
solution = hill_climbing(block_world)
print("\nFinal Solution:", solution)

```

OUTPUT:

```
Initial State: ['b', 'c', 'd', 'a']
Goal State: ['a', 'b', 'c', 'd']
Step 0: ['b', 'c', 'd', 'a'], Cost: 0
Step 1: ['c', 'b', 'd', 'a'], Cost: 1
Step 2: ['d', 'b', 'c', 'a'], Cost: 2
Step 3: ['a', 'b', 'c', 'd'], Cost: 4

Final Solution: ['a', 'b', 'c', 'd']
```

DATE	23 – 07 – 2025
PROGRAM NO.: 05	Program to implement Minimax Algorithm

SOURCE CODE:

```

# Hardcoded leaf values (bottom of tree)
leaf_values = [3, 5, 2, 9, 12, 5, 23, 23] # 8 leaf nodes

# Minimax Algorithm
def minimax(depth, index, is_max, values):
    # Base case: if at leaf node
    if depth == 3:
        return values[index]

    # If maximizer's turn
    if is_max:
        left = minimax(depth + 1, index * 2, False, values)
        right = minimax(depth + 1, index * 2 + 1, False, values)
        return max(left, right)

    # If minimizer's turn
    else:
        left = minimax(depth + 1, index * 2, True, values)
        right = minimax(depth + 1, index * 2 + 1, True, values)
        return min(left, right)

# Root call (MAX player)
best_value = minimax(0, 0, True, leaf_values)

# Output
print("Leaf values:", leaf_values)
print("Best value according to Minimax:", best_value)

```

OUTPUT:

```
Leaf values: [3, 5, 2, 9, 12, 5, 23, 23]  
Best value according to Minimax: 12  
,
```

DATE	23 – 07 – 2025
PROGRAM NO.: 06	Program to implement Alpha Beta Pruning

SOURCE CODE:

```

def alpha_beta_pruning(values, depth, alpha, beta, index, is_maximizing, visited):
    if depth == 0:
        visited.append(values[index])
        return values[index]

    if is_maximizing:
        best = float('-inf')
        for i in range(2):
            val = alpha_beta_pruning(values, depth - 1, alpha, beta, index * 2 + i, False, visited)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = float('inf')
        for i in range(2):
            val = alpha_beta_pruning(values, depth - 1, alpha, beta, index * 2 + i, True, visited)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best

# Input
depth = int(input("Enter tree depth (e.g., 3): "))
leaf_count = 2 ** depth
values = list(map(int, input(f"Enter {leaf_count} leaf values: ").split()))

```

```
if len(values) != leaf_count:
    print(f"Error: You must enter exactly {leaf_count} values.")
else:
    visited = []
    result = alpha_beta_pruning(values, depth, float('-inf'), float('inf'), 0, True, visited)
    print("\nOptimal value:", result)
    print("Nodes visited:", visited)
    print("Nodes pruned:", set(values) - set(visited))
```

OUTPUT:

```
Enter tree depth (e.g., 3): 3
Enter 8 leaf values: 2 4 6 8 1 3 5 7

Optimal value: 4
Nodes visited: [2, 4, 6, 1, 3]
Nodes pruned: {8, 5, 7}
```

DATE	23 – 07 – 2025
PROGRAM NO.: 07	Program to implement N-Queens problem

SOURCE CODE:

```
def print_board(board):
    for row in board:
        print(" ".join("Q" if col else "." for col in row))
    print()

def is_safe(board, row, col, n):
    # Check this column on the upper side
    for i in range(row):
        if board[i][col]:
            return False

    # Check upper left diagonal
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1

    # Check upper right diagonal
    i, j = row, col
    while i >= 0 and j < n:
        if board[i][j]:
            return False
        i -= 1
        j += 1

    return True
```

```
def solve_n_queens_util(board, row, n, solutions):
    if row == n:
        # Deep copy of board configuration
        solution = ["Q" if col else "." for col in r] for r in board
        solutions.append(solution)
        return

    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = True
            solve_n_queens_util(board, row + 1, n, solutions)
            board[row][col] = False # backtrack

def solve_n_queens(n):
    board = [[False for _ in range(n)] for _ in range(n)]
    solutions = []
    solve_n_queens_util(board, 0, n, solutions)
    return solutions

# Example usage
n = 4 # You can change this value
solutions = solve_n_queens(n)
print(f"Total solutions for {n}-Queens: {len(solutions)}")
for sol in solutions:
    for row in sol:
        print(" ".join(row))
    print()
```

OUTPUT:

```
Total solutions for 4-Queens: 2
```

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

```
. . Q .  
Q . . .  
. . . Q  
. Q . .
```

DATE	22 – 08 – 2025
PROGRAM NO.: 08	Program to implement Water-Jug Problem

SOURCE CODE:

```

from collections import deque

def water_jug_bfs(jug1, jug2, target):
    visited = set()
    queue = deque()

    # Start from (0, 0)
    queue.append((0, 0))
    visited.add((0, 0))

    while queue:
        x, y = queue.popleft()
        print(f"Visited state: X = {x}L, Y = {y}L")

        # Check if goal is reached
        if x == target or y == target:
            print("Goal reached!")
            return True

        # All possible next states
        next_states = set()

        # Fill Jug1 or Jug2
        next_states.add((jug1, y))
        next_states.add((x, jug2))

        # Empty Jug1 or Jug2
        next_states.add((0, y))
        next_states.add((x, 0))

```

```
# Pour Jug1 -> Jug2
pour = min(x, jug2 - y)
next_states.add((x - pour, y + pour))

# Pour Jug2 -> Jug1
pour = min(y, jug1 - x)
next_states.add((x + pour, y - pour))

# Process new states
for state in next_states:
    if state not in visited:
        visited.add(state)
        queue.append(state)

print("No solution found.")
return False

# Example usage
jug1_capacity = 4
jug2_capacity = 3
target = 2

water_jug_bfs(jug1_capacity, jug2_capacity, target)
```

OUTPUT:

```
Visited state: X = 0L, Y = 0L
Visited state: X = 4L, Y = 0L
Visited state: X = 0L, Y = 3L
Visited state: X = 1L, Y = 3L
Visited state: X = 4L, Y = 3L
Visited state: X = 3L, Y = 0L
Visited state: X = 1L, Y = 0L
Visited state: X = 3L, Y = 3L
Visited state: X = 0L, Y = 1L
Visited state: X = 4L, Y = 2L
Goal reached!
```

DATE	22 – 08 – 2025
PROGRAM NO.: 09	Program to implement Cryptarithmic problem

SOURCE CODE:

```

import itertools

def solve_manual_cryptarithmic():
    # Input words from user
    word1 = input("Enter the first word: ").strip().upper()
    word2 = input("Enter the second word: ").strip().upper()
    result = input("Enter the result word: ").strip().upper()

    # Extract unique letters
    letters = set(word1 + word2 + result)
    if len(letters) > 10:
        print("Too many unique letters (maximum is 10).")
        return

    letters = list(letters)
    leading_letters = {word1[0], word2[0], result[0]}

    for perm in itertools.permutations(range(10), len(letters)):
        letter_to_digit = dict(zip(letters, perm))

        # Skip if any leading letter is 0
        if any(letter_to_digit[char] == 0 for char in leading_letters):
            continue

        def word_to_number(word):
            return int("".join(str(letter_to_digit[c]) for c in word))

        num1 = word_to_number(word1)
        num2 = word_to_number(word2)

```

```

res = word_to_number(result)

if num1 + num2 == res:
    print(f"\n✓ Solution Found!")
    print(f"{word1} = {num1}")
    print(f"{word2} = {num2}")
    print(f"{result} = {res}")
    print(f"Letter Mapping: {letter_to_digit}")
    return

print("✗ No solution found.")

# Run the program
solve_manual_cryptarithmic()

```

OUTPUT:

```

Enter the first word: send
Enter the second word: more
Enter the result word: money

```

```

☒ Solution Found!
SEND = 9567
MORE = 1085
MONEY = 10652
Letter Mapping: {'Y': 2, 'R': 8, 'S': 9, 'O': 0, 'E': 5, 'M': 1, 'D': 7, 'N': 6}

```

```

Enter the first word: cross
Enter the second word: roads
Enter the result word: danger

```

```

☒ Solution Found!
CROSS = 96233
ROADS = 62513
DANGER = 158746
Letter Mapping: {'G': 7, 'A': 5, 'R': 6, 'O': 2, 'E': 4, 'S': 3, 'C': 9, 'N': 8, 'D': 1}

```

DATE	22 – 08 – 2025
PROGRAM NO.: 10	Implementation of Text-Preprocessing using NLTK (Tokenization, Stemming, Lemmatization and Stop words Removal)

SOURCE CODE:

```

import nltk
nltk.download("punkt")
nltk.download("stopwords")
nltk.download("wordnet")

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

text = "NLTK helps with tokenization, stemming, and lemmatization."
tokens = word_tokenize(text)
filtered = [w for w in tokens if w.isalpha() and w.lower() not in
set(stopwords.words("english"))]
stems = [PorterStemmer().stem(w) for w in filtered]
lemmas = [WordNetLemmatizer().lemmatize(w) for w in filtered]

print(tokens)
print(filtered)
print(stems)
print(lemmas)

# Install nltk (only once in Colab or first time on PC)
# !pip install nltk
import nltk
nltk.download("punkt")
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("punkt_tab") # Added to fix LookupError

```

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Example text
text = "NLTK is a great library. It helps in processing words like running, runs, and ran."

# 1. Tokenization
tokens = word_tokenize(text)
print("Tokens:", tokens)

# 2. Stopword Removal
stop_words = set(stopwords.words("english"))
filtered = [w for w in tokens if w.lower() not in stop_words and w.isalpha()]
print("After Stopword Removal:", filtered)

# 3. Stemming
stemmer = PorterStemmer()
stems = [stemmer.stem(w) for w in filtered]
print("Stems:", stems)

# 4. Lemmatization
lemmatizer = WordNetLemmatizer()
lemmas = [lemmatizer.lemmatize(w) for w in filtered]
print("Lemmas:", lemmas)
```

OUTPUT:

```
['NLTK', 'helps', 'with', 'tokenization', ',', 'stemming', ',', 'and', 'lemmatization', '.']  
['NLTK', 'helps', 'tokenization', 'stemming', 'lemmatization']  
['nltk', 'help', 'token', 'stem', 'lemmat']  
['NLTK', 'help', 'tokenization', 'stemming', 'lemmatization']
```

```
Tokens: ['NLTK', 'is', 'a', 'great', 'library', '.', 'It', 'helps', 'in', 'processing', 'words', 'like', 'running', ',', 'runs', ',', 'and', 'ran', '.']  
After Stopword Removal: ['NLTK', 'great', 'library', 'helps', 'processing', 'words', 'like', 'running', 'runs', 'ran']  
Stems: ['nltk', 'great', 'librari', 'help', 'process', 'word', 'like', 'run', 'run', 'ran']  
Lemmas: ['NLTK', 'great', 'library', 'help', 'processing', 'word', 'like', 'running', 'run', 'ran']
```

DATE	22 – 08 – 2025
PROGRAM NO.: 11	Build a rule-based chatbot that responds to user input with predefined answers

SOURCE CODE:

```

from transformers import BlenderbotTokenizer, BlenderbotForConditionalGeneration

model_name = "facebook/blenderbot-400M-distill"
tokenizer = BlenderbotTokenizer.from_pretrained(model_name)
model = BlenderbotForConditionalGeneration.from_pretrained(model_name)

print("Chatbot : Hello! Type 'bye' to exit.")

while True:
    user_input = input("You: ")
    if user_input.lower() == "bye":
        print("Chatbot : Goodbye ")
        break

    inputs = tokenizer([user_input], return_tensors="pt")
    reply_ids = model.generate(**inputs)
    response = tokenizer.decode(reply_ids[0], skip_special_tokens=True)
    print("Chatbot :", response)

```

OUTPUT:

```

Chatbot : Hello! Type 'bye' to exit.
You: hi
Chatbot : Hi! How are you? I just got back from walking my dog. Do you have any pets?
You: no tell me about your pet
Chatbot : I have a dog and a cat. I love them both so much. Do you have any pets?
You: no i dont have pet but i like dogs though
Chatbot : Dogs are great. They are man's best friend for sure. What kind of dog do you have?
You: pug
Chatbot : I have a pug. They are a small, short-haired breed of dog.
You: bye
Chatbot : Goodbye

```

